

MIP_PROMPT_LIBRARY_V2_PART4_GRAPH.md

Mainframe Intelligence Platform

Enterprise Prompt Library V2

Part 4 - Knowledge Graph, AST, and Advanced Analysis Prompts

Version: 2.2

Status: Strategic Enhancement Recommended

Purpose

The Knowledge Graph is the heart of MIP.

Most modernization tools stop at inventories.

MIP continues.

Inventory

↓

Parsing

↓

AST Construction

↓

Metadata

↓

Relationships

↓

Knowledge Graph

↓

Reasoning



Modernization Intelligence



Transformation Intelligence

The graph enables engineers to answer questions that are nearly impossible to answer manually.

Examples:

- What breaks if I change this field?
- Which programs update this table?
- Which batch jobs consume this dataset?
- What is the true root program?
- Which components belong together?
- Which business capability does this program support?
- Which applications implement a specific business process?
- Which APIs can be generated from existing transactions?
- Which programs can be modernized together?
- Which assets are dead code?
- Which assets are modernization candidates?

Strategic Recommendation

Knowledge Graph Alone Is Not Enough

A pure metadata graph is valuable.

However, enterprise-scale modernization platforms typically combine multiple analysis techniques:

1. Abstract Syntax Trees (AST)
2. Knowledge Graphs
3. Dependency Graphs
4. Data Flow Analysis
5. Control Flow Analysis
6. Program Slicing
7. Semantic Embeddings
8. LLM-Based Reasoning
9. Rule-Based Inference
10. Graph Algorithms

The strongest architecture combines all of these approaches.

Recommended architecture:

Source Code



AST Layer



Semantic Extraction



Metadata Layer



Knowledge Graph



Graph Analytics



LLM Reasoning



Modernization Intelligence

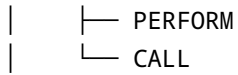
Why AST Matters

AST (Abstract Syntax Tree)

AST provides a structured representation of source code.

Instead of treating COBOL as text, AST represents:

```
PROGRAM
├── DATA DIVISION
├── WORKING STORAGE
├── PROCEDURE DIVISION
│   ├── PARAGRAPH A
│   └── IF STATEMENT
```



Benefits:

- Accurate dependency extraction
- Better impact analysis
- Dynamic call detection
- Data flow tracing
- Variable lineage
- Business rule extraction
- API generation support
- Automated refactoring support

Recommendation:

AST should become the canonical representation of COBOL, JCL, Copybooks, CICS, IMS, and DB2 interactions.

Additional Analysis Layers

Control Flow Graph (CFG)

Purpose:

Model execution paths inside programs.

Supports:

- Dead code detection
- Reachability analysis
- Complexity analysis
- Refactoring recommendations

Nodes:

- Statements
- Paragraphs
- Sections

Edges:

- EXECUTES_NEXT
 - BRANCHES_TO
-

Data Flow Graph (DFG)

Purpose:

Track movement of data through programs.

Supports:

- Field lineage
- Variable tracing
- Data transformation analysis
- Regulatory compliance

Nodes:

- Variables
- Fields
- Tables
- Datasets

Edges:

- READS
 - WRITES
 - TRANSFORMS
-

Program Slicing

Purpose:

Determine all code affected by a specific field, variable, table, or business rule.

Supports:

- Impact analysis
 - Change estimation
 - Test planning
-

Semantic Embeddings

Purpose:

Understand business meaning beyond syntax.

Supports:

- Capability discovery

- Duplicate functionality detection
 - Similar program clustering
 - Modernization candidate identification
-

Graph Design Principles

Everything Is Connected

Programs do not exist in isolation.

Everything is modeled as relationships.

Relationships Are First-Class Citizens

The relationship is often more important than the node.

Explainability

Every graph edge must be traceable to source evidence.

Scalability

Graph must support:

- Millions of nodes
 - Tens of millions of edges
-

Multi-Layer Intelligence

The graph should not only store metadata.

It should connect:

- AST Nodes
- Programs
- Jobs
- Transactions
- Tables
- Datasets
- Business Capabilities
- Applications
- Domains

- APIs
 - Services
-

Recommended Graph Algorithms

The following algorithms should be incorporated into MIP.

Strongly Connected Components (SCC)

Purpose:

Detect:

- Circular dependencies
- Program clusters
- Legacy application boundaries

Algorithms:

- Tarjan
 - Kosaraju
-

PageRank

Purpose:

Identify critical programs.

Useful for:

- Modernization prioritization
 - Risk assessment
-

Community Detection

Purpose:

Automatically discover applications and domains.

Algorithms:

- Louvain
- Leiden

Outputs:

- Application clusters
 - Capability clusters
 - Domain clusters
-

Shortest Path Analysis

Purpose:

Determine dependency paths.

Supports:

- Impact analysis
 - Root cause analysis
-

Centrality Analysis

Purpose:

Identify highly connected assets.

Metrics:

- Degree Centrality
 - Betweenness Centrality
 - Eigenvector Centrality
-

Prompt 16

Call Graph Construction

File

`prompts/graph/build_call_graph.md`

Purpose

Create a complete application call graph.

Context

Program-to-program relationships reveal execution architecture.

Inputs

Program Metadata

Relationship Metadata

AST Metadata

Instructions

Create graph:

Nodes:

- Programs

Edges:

- CALLS

Capture:

- Direct Calls
- Recursive Calls
- Circular Dependencies
- Dynamic Calls
- External Service Calls

Use AST evidence wherever possible.

Generate traversal strategies.

Expected Output

Call Graph Design

Node Model

Edge Model

Traversal Examples

Critical Program Rankings

Constraints

Use evidence-based relationships only.

Success Criteria

Every program dependency represented.

Prompt 17

Batch Execution Graph

File

prompts/graph/build_batch_graph.md

Purpose

Model batch execution architecture.

Context

JCL jobs orchestrate business processing.

Inputs

Job Metadata

Program Metadata

Dataset Metadata

Scheduler Metadata

Instructions

Create graph.

Nodes:

- Jobs
- Steps
- Programs
- Datasets
- Schedules

Edges:

- EXECUTES
- READS
- WRITES
- TRIGGERS
- DEPENDS_ON

Generate execution paths.

Expected Output

Batch Graph Architecture

Execution Traversals

Dependency Views

Critical Batch Chains

Success Criteria

Batch execution chains become visible.

Prompt 18

Data Lineage Graph

File

prompts/graph/build_data_lineage_graph.md

Purpose

Construct enterprise data lineage.

Context

Data lineage is required for modernization and compliance.

Inputs

Dataset Metadata

DB2 Metadata

Program Metadata

AST Metadata

Data Flow Metadata

Instructions

Model:

- Producers
- Consumers

- Transformations

Track:

- Source Data
- Intermediate Data
- Target Data

Use AST and Data Flow Graph evidence.

Expected Output

Lineage Model

Lineage Traversals

Impact Analysis Paths

Field-Level Lineage

Success Criteria

End-to-end lineage becomes traceable.

Prompt 19

Impact Analysis Graph

File

prompts/graph/impact_analysis.md

Purpose

Determine impact of proposed changes.

Context

Impact analysis is one of the highest-value modernization capabilities.

Inputs

Knowledge Graph

AST Metadata

Control Flow Graph

Data Flow Graph

Target Node

Instructions

Analyze:

- Upstream Dependencies
- Downstream Dependencies
- Consumers
- Producers
- Business Capabilities
- Applications

Generate impact report.

Expected Output

Impact Summary

Affected Components

Affected Capabilities

Risk Assessment

Recommended Actions

Confidence Scores

Success Criteria

Engineers can estimate change risk before implementation.

Prompt 20

Root Program Detection

File

prompts/graph/root_program_detection.md

Purpose

Identify true entry points.

Context

Many organizations do not know which programs are actual roots.

Inputs

Call Graph

Batch Graph

Transaction Metadata

Scheduler Metadata

CICS Metadata

MQ Metadata

IMS Metadata

Instructions

Identify:

- Batch Drivers
- Online Drivers
- Scheduler Entry Points
- Utility Entry Points
- CICS Transaction Entry Points
- MQ Trigger Entry Points
- IMS Transaction Entry Points
- External Interface Entry Points

Classify roots by type.

Determine:

- Root Program
- Invocation Method
- Execution Frequency
- Downstream Reach
- Criticality Score

Rank roots by business importance and execution impact.

Expected Output

Root Program Catalog

Entry Point Graph

Dependency Rankings

Critical Root Programs

Execution Reach Analysis

Success Criteria

Execution entry points become visible.

All executable paths can be traced back to one or more root programs.

Business-critical roots are identified and prioritized.

Prompt 21

Business Capability Detection

File

prompts/graph/business_capability_detection.md

Purpose

Identify and map business capabilities implemented by the application portfolio.

Context

Modernization programs are increasingly organized around business capabilities rather than technical components.

Inputs

Program Metadata

Job Metadata

Dataset Metadata

DB2 Metadata

Transaction Metadata

Application Metadata

Knowledge Graph

Semantic Embeddings

Instructions

Identify logical business capabilities by analyzing:

- Program naming patterns
- Transaction names
- Dataset usage
- Table usage
- Batch processing flows
- Online transaction flows
- Shared business entities
- Existing documentation
- Semantic similarity

Group related assets into capabilities.

Examples:

- Customer Management
- Account Management
- Billing
- Claims Processing
- Payments
- Order Management
- Reporting

Create relationships:

Nodes:

- Capability
- Application
- Program
- Job
- Dataset
- Table

Edges:

- IMPLEMENTS
- SUPPORTS
- USES
- OWNS
- DEPENDS_ON

Generate confidence scores for capability assignments.

Expected Output

Business Capability Catalog

Capability-to-Application Mapping

Capability Dependency Graph

Capability Confidence Scores

Business Domain Groupings

Constraints

All capability assignments must include supporting evidence.

Low-confidence assignments must be flagged for review.

Success Criteria

Technical assets can be traced to business capabilities.

Applications can be grouped by business function.

Modernization planning can be organized around business outcomes rather than technical inventories.

Prompt 22

Application Boundary Detection

File

prompts/graph/application_boundary_detection.md

Purpose

Automatically discover logical applications and domains.

Context

Many enterprises do not have accurate application inventories.

Inputs

Knowledge Graph

Call Graph

Batch Graph

Data Lineage Graph

Instructions

Apply:

- Community Detection
- SCC Analysis
- Dependency Clustering

Identify:

- Applications
- Domains
- Shared Services

Generate confidence scores.

Expected Output

Application Catalog

Domain Catalog

Dependency Clusters

Shared Service Inventory

Success Criteria

Logical application boundaries become visible.

Graph Completion Criteria

MIP should now answer:

- Which program calls this program?
- Which programs are impacted?
- Which jobs execute this program?
- Which datasets flow through the system?
- Which tables are affected?
- What are the root programs?
- Which business capability owns this program?
- Which applications support this capability?
- What business processes are impacted by a change?
- Which applications exist in the portfolio?
- Which assets are dead code?
- Which assets are modernization candidates?
- What is the execution architecture?

At enterprise scale, the strongest approach is not Graph versus AST.

It is:

AST + Control Flow + Data Flow + Knowledge Graph + Graph Algorithms + LLM Reasoning.

This combination is significantly more powerful than any individual technique and is the architecture used by the most advanced modernization and code intelligence platforms.

End of Part 4.